

MPC 控制器原理说明

从零开始理解 Local MPC、ADMM MPC 与 soft/hard 模式

MADRL_ESS 项目理解笔记

2026 年 7 月 27 日

目录

1	先不看公式：MPC 到底在做什么	3
2	优化问题是什么：给完全没学过算法的读者	3
2.1	决策变量：控制器手里的旋钮	3
2.2	约束条件：物理世界的边界	4
2.3	目标函数：什么叫“最好”	4
2.4	读 MPC 公式的小技巧	5
3	MPC 和 Rule-based 的区别	5
4	本项目中的两种 MPC	5
5	符号说明	6
6	Local MPC：每个用户自己规划	6
6.1	Local MPC 一步一步做什么	6
6.2	为什么电池要拆成充电和放电	7
6.3	静态电池约束	7
6.4	PV 在 MPC 中如何处理	8
6.5	EV 约束	8
6.6	局部功率平衡	9
6.7	Local MPC 的目标函数	10
6.8	Local MPC 的优点和局限	10
7	ADMM MPC：让多个用户边自私边协调	10
7.1	为什么需要 ADMM	10

7.2	先用故事理解 ADMM	11
7.3	ADMM 中三个新概念	11
7.4	被协调的量：净负荷贡献	11
7.5	局部子问题：每个 agent 怎样被协调	12
7.6	聚合投影：把总功率拉回安全范围	12
7.7	对偶变量更新：记住偏差	13
7.8	什么时候停止迭代	13
7.9	第一步动作投影：为什么还要再修一次	13
7.10	ADMM MPC 的优点和局限	14
8	soft 和 hard 模式：到底软在哪里，硬在哪里	14
8.1	先给核心直觉	14
8.2	soft 模式的数学表达	15
8.3	hard 模式在本项目 MPC 中的实现	15
8.4	hard projection 怎么算	16
8.5	soft 和 hard 行为差异	16
9	Local MPC 和 ADMM MPC 在 soft/hard 下的对比	17
10	Global MISOCP 和两种 MPC 的关系	17
11	从零基础角度总结	18

1 先不看公式：MPC 到底在做什么

MPC 的全称是 Model Predictive Control，中文常叫模型预测控制。这个名字里有三个关键词：

- Model：我知道系统大致会怎样变化，例如电池充电后 SoC 会升高，放电后 SoC 会降低；
- Predictive：我有未来一小段时间的预测，例如未来电价、负荷、PV 发电；
- Control：我现在要决定动作，例如电池充不充、EV 充不充、PV 要不要弃光。

如果用一句非常直白的话概括：

MPC 每一步都先“看一眼未来”，算一份未来计划，但真正执行时只执行计划的第一步。

为什么只执行第一步？因为预测会变。下一步到来时，系统状态也变了，新的预测也可能不同。MPC 不会死守旧计划，而是重新规划。这种方法叫滚动时域控制。

可以把它想成开车导航。导航会规划未来几十公里，但你真正做的只是当前路口左转还是直行。等你开到下一个路口，导航会根据新路况重新规划。MPC 对电池、EV、PV 做的事情也很像：它规划未来多个时间步，但只把当前一步动作交给环境执行。

2 优化问题是什么：给完全没学过算法的读者

MPC 的核心不是神秘的“智能”，而是每一步解一个优化问题。优化问题可以理解为：

在一堆允许的选择里，找一个最好的选择。

一个优化问题通常由三部分组成：

1. 决策变量：控制器可以选择的量；
2. 约束条件：选择时不能违反的规则；
3. 目标函数：用来判断哪个选择更好的打分函数。

2.1 决策变量：控制器手里的旋钮

在本项目中，MPC 可以决定：

$$c_{i,k}, \quad d_{i,k}, \quad u_{i,k}, \quad e_{i,k}. \quad (1)$$

它们分别表示：

- $c_{i,k}$: 第 i 个用户在未来第 k 步给静态电池充多少电;
- $d_{i,k}$: 第 i 个用户在未来第 k 步让静态电池放多少电;
- $u_{i,k}$: 第 i 个用户在未来第 k 步丢掉多少 PV 发电;
- $e_{i,k}$: 第 i 个用户在未来第 k 步给 EV 充多少电。

这些量叫决策变量，是因为它们不是已经发生的事实，而是控制器要做出的决定。

2.2 约束条件：物理世界的边界

约束条件就是不能违反的规则。例如：

$$0 \leq e_{i,k} \leq P_i^{ev,max}. \quad (2)$$

这句话的意思是：EV 充电功率不能小于 0，也不能超过充电桩和车辆允许的最大功率。约束不是建议，而是物理边界。优化器不能为了省钱或好看而让 EV 充电功率变成负数，也不能让电池能量超过容量。

如果所有约束能同时满足，我们说这个问题是可行的。如果约束互相冲突，例如 EV 只剩 15 分钟就要离家，但要求从 10% 充到 90%，最大充电功率又不够，那么问题可能不可行。

2.3 目标函数：什么叫“最好”

目标函数是优化器用来比较不同计划的分数。MPC 一般写成：

$$\min J. \quad (3)$$

这里 $\min$ 的意思是 minimize，也就是让 J 尽可能小。 J 可以是电费、惩罚、设备磨损等多个项目加起来的总成本。

例如，如果电价是 $\tilde{p}_t$ 欧元/kWh，某一步充电功率是 P_t kW，时间长度是 Δt 小时，那么这一小步买电费用近似为：

$$\text{费用} = \tilde{p}_t P_t \Delta t. \quad (4)$$

单位也能对上：

$$\frac{\text{欧元}}{\text{kWh}} \times \text{kW} \times \text{h} = \text{欧元}. \quad (5)$$

所以优化器不是“凭感觉”省钱，而是在每个未来时间步计算成本，然后找总成本最低的一组动作。

2.4 读 MPC 公式的小技巧

初学者看到公式容易紧张。其实读 MPC 公式可以按这个顺序：

1. 先看变量单位：kW 是功率，kWh 是能量，欧元/kWh 是价格；
2. 再看等式左右：等式左右应该表示同一类物理量；
3. 最后看正负号：充电通常增加负荷，放电通常减少负荷，弃光会减少可用 PV。

如果公式能解释成一句物理语言，它通常就不难。例如：

$$E_{k+1} = E_k + \eta P \Delta t \quad (6)$$

就是“下一步能量 = 当前能量 + 这一小段时间充进去的能量”。

3 MPC 和 Rule-based 的区别

Rule-based 控制器像固定规则表。例如：

如果电价低，就充电；如果电价高，就放电。

这种方法容易理解，但它通常只看当前信息或简单阈值。MPC 不同，它会看未来。例如：

现在电价有点低，但两个小时后再更低，所以先不充；等两个小时后再充。

因此，MPC 的核心优势是“有计划”。它不是只问“现在该怎么办”，而是问：

如果我现在这样做，接下来几个小时会发生什么？

4 本项目中的两种 MPC

本项目主要有两种 MPC：

- Local MPC：每个 agent 独立做自己的优化；
- ADMM MPC：每个 agent 还是做自己的优化，但大家通过 ADMM 协调总变压器功率。

可以用一个类比理解：

- Local MPC 像每户家庭各自省钱，互相不商量；
- ADMM MPC 像每户家庭先各自做计划，然后小区协调员检查总功率是否超过变压器限制，超了就让大家改一点。

仓库中还存在 Global MISOCP 基线。它像一个中央调度器，直接把全网物理约束放进一个大优化问题。它更完整，但求解更重。本文件重点讲 Local MPC 和 ADMM MPC。

5 符号说明

符号	单位	含义
i	无	agent 编号, 本项目默认有多个家庭或节点
k	无	MPC 预测窗口内的步数, $k = 0, \dots, H-1$
t	无	环境当前真实时间步
H	步	MPC 向未来看的步数, 叫 prediction horizon
Δt	小时	一个时间步的长度, 默认 0.25 小时
$\tilde{p}_{t+k}$	欧元/kWh	未来第 k 步进口电价
$L_{i,t+k}$	kW	第 i 个用户未来第 k 步负荷预测
$G_{i,t+k}$	kW	第 i 个用户未来第 k 步 PV 可用功率预测
$c_{i,k}$	kW	静态电池充电功率, 非负
$d_{i,k}$	kW	静态电池放电功率, 非负
$P_{i,k}^b$	kW	有符号电池功率, $P_{i,k}^b = c_{i,k} - d_{i,k}$
$u_{i,k}$	kW	PV 弃光功率
$e_{i,k}$	kW	EV 充电功率
$E_{i,k}^b$	kWh	静态电池能量状态
$E_{i,k}^{ev}$	kWh	EV 电池能量状态
C_i^b	kWh	静态电池容量
C_i^{ev}	kWh	EV 电池容量
$P_i^{b,\max}$	kW	静态电池最大充放电功率
$P_i^{ev,\max}$	kW	EV 最大充电功率
η_b, η_{ev}	无	静态电池效率和 EV 充电效率

注意: 如果当前真实时间是 t , 那么预测窗口内的 $k = 0$ 就是现在这一小步, $k = 1$ 是下一小步, $k = 2$ 是再下一小步。

6 Local MPC: 每个用户自己规划

6.1 Local MPC 一步一步做什么

Local MPC 的代码入口是 `controllers/mpc_local.py`。每个环境步, 它大致做五件事:

1. 读取当前状态: 电池 SoC、EV SoC、EV 是否在家;
2. 读取未来预测: 未来电价、负荷、PV 可用功率;

3. 建立优化问题：把可选动作、物理约束和成本函数写进去；
4. 调用求解器：让求解器找出未来 H 步的低成本计划；
5. 只执行第一步：把 $k = 0$ 的动作交给环境，下一步再重新算。

它的输入可以概括为：

$$\{\tilde{p}_{t:t+H-1}, L_{i,t:t+H-1}, G_{i,t:t+H-1}\}. \quad (7)$$

它要求解的动作序列是：

$$\{c_{i,k}, d_{i,k}, u_{i,k}, e_{i,k}\}_{k=0}^{H-1}. \quad (8)$$

求解完成后，真正送给环境执行的是第一步：

$$P_{i,t}^b = c_{i,0} - d_{i,0}, \quad G_{i,t}^{curt} = u_{i,0}, \quad P_{i,t}^{ev} = e_{i,0}. \quad (9)$$

这里要特别记住：MPC 虽然算了未来 H 步，但不会一次性全部执行。它只执行当前动作，然后下一步重新预测和优化。

6.2 为什么电池要拆成充电和放电

环境里的电池功率可以用一个有正负号的量表示：

$$P_{i,k}^b > 0 \text{ 表示充电}, \quad P_{i,k}^b < 0 \text{ 表示放电}. \quad (10)$$

但很多优化建模更喜欢非负变量，所以 Local MPC 把电池功率拆成两个变量：

$$P_{i,k}^b = c_{i,k} - d_{i,k}. \quad (11)$$

其中：

$$c_{i,k} \geq 0, \quad d_{i,k} \geq 0. \quad (12)$$

如果 $c_{i,k} = 5, d_{i,k} = 0$ ，就是充电 5 kW。如果 $c_{i,k} = 0, d_{i,k} = 5$ ，就是放电 5 kW。这样写的好处是每个变量都有明确范围，求解器更容易处理。

6.3 静态电池约束

静态电池能量更新为：

$$E_{i,k+1}^b = E_{i,k}^b + \eta_b c_{i,k} \Delta t - \frac{d_{i,k} \Delta t}{\eta_b}. \quad (13)$$

逐项解释：

- $E_{i,k}^b$ ：当前电池里有多少能量；
- $\eta_b c_{i,k} \Delta t$ ：这一小步充进去的有效能量；

- $\frac{d_{i,k}\Delta t}{\eta_b}$: 这一小步放电导致电池内部减少的能量;
- $\eta_b < 1$ 表示充放电有损耗。

SoC 和能量的关系为:

$$E_{i,k}^b = SOC_{i,k}^b C_i^b. \quad (14)$$

例如容量 $C_i^b = 10$ kWh, SoC 为 50%, 那么电池能量就是 5 kWh。

电池能量不能超过上下界:

$$SOC_{\min}^b C_i^b \leq E_{i,k}^b \leq SOC_{\max}^b C_i^b. \quad (15)$$

充放电功率也不能超过最大值:

$$0 \leq c_{i,k} \leq P_i^{b,\max}, \quad 0 \leq d_{i,k} \leq P_i^{b,\max}. \quad (16)$$

这些约束的物理意义是: 电池不能充爆, 不能放空, 也不能瞬间以超过硬件能力的功率充放电。

6.4 PV 在 MPC 中如何处理

PV 表示光伏发电。对 MPC 来说, 未来 PV 不是控制器能决定的, 而是预测输入:

$$G_{i,t+k}. \quad (17)$$

也就是说, MPC 不能命令太阳多发电或少发电。它只能决定: 如果预测有这么多 PV, 哪些用掉, 哪些丢掉。丢掉的部分叫弃光, 变量是:

$$u_{i,k}. \quad (18)$$

弃光范围为:

$$0 \leq u_{i,k} \leq \max(G_{i,t+k}, 0). \quad (19)$$

实际使用的 PV 功率是:

$$G_{i,k}^{use} = G_{i,t+k} - u_{i,k}. \quad (20)$$

如果 $u_{i,k} = 0$, 表示 PV 全部使用。如果 $u_{i,k} = 2$ kW, 表示有 2 kW 光伏没有进入负荷、电池或电网, 而是被主动削减。弃光本身不是好事, 但在某些情况下它能帮助满足功率平衡或避免不希望的出口。

6.5 EV 约束

EV 充电功率为 $e_{i,k}$ 。如果 EV 在家:

$$0 \leq e_{i,k} \leq P_i^{ev,\max}. \quad (21)$$

如果 EV 不在家:

$$e_{i,k} = 0. \quad (22)$$

这不是算法偏好, 而是现实条件: 车不在家时, 家用充电桩无法给车充电。

EV 能量更新为:

$$E_{i,k+1}^{ev} = E_{i,k}^{ev} + \eta_{ev} e_{i,k} \Delta t. \quad (23)$$

EV 能量和 SoC 的关系为:

$$E_{i,k}^{ev} = SOC_{i,k}^{ev} C_i^{ev}. \quad (24)$$

EV 能量也有上下界:

$$SOC_{\min}^{ev} C_i^{ev} \leq E_{i,k}^{ev} \leq SOC_{\max}^{ev} C_i^{ev}. \quad (25)$$

6.6 局部功率平衡

对一个用户来说, 调度后的净负荷可以写成:

$$n_{i,k} = L_{i,t+k} - G_{i,t+k} + u_{i,k} + c_{i,k} - d_{i,k} + e_{i,k}. \quad (26)$$

这个公式很重要, 可以逐项读:

- L : 家庭负荷, 要用电, 所以增加净负荷;
- $-G$: PV 发电可以抵消负荷, 所以减少净负荷;
- $+u$: 弃掉 PV 后, 可用光伏变少, 所以净负荷增加;
- $+c$: 电池充电要吸收电力, 所以增加净负荷;
- $-d$: 电池放电供给家庭, 所以减少净负荷;
- $+e$: EV 充电要用电, 所以增加净负荷。

如果 $n_{i,k} > 0$, 用户从电网买电。如果 $n_{i,k} < 0$, 用户向电网送电。

代码中还会用进口变量 $P_{i,k}^{imp}$ 和出口变量 $P_{i,k}^{exp}$ 表示:

$$P_{i,k}^{imp} - P_{i,k}^{exp} = n_{i,k}. \quad (27)$$

同时会用二进制变量避免同一时间既进口又出口。直觉上就是: 一个用户同一瞬间不应该既买电又卖电, 否则数学上可能出现非物理的套利解。

6.7 Local MPC 的目标函数

Local MPC 的目标是最小化未来窗口内的总成本:

$$\min J_i^{local}. \quad (28)$$

本项目中的主要形式可以写成:

$$J_i^{local} = \sum_{k=0}^{H-1} [\tilde{p}_{t+k} \Delta t c_{i,k} - \tilde{p}_{t+k} \Delta t d_{i,k} + \tilde{p}_{t+k} \Delta t e_{i,k} + \epsilon_b \Delta t (c_{i,k} + d_{i,k})] + w_{dep}^{ev} g_i^2. \quad (29)$$

逐项解释:

- $\tilde{p} \Delta t c$: 电池充电需要买电, 是成本;
- $-\tilde{p} \Delta t d$: 电池放电可以抵消高价购电, 所以在成本里是负号;
- $\tilde{p} \Delta t e$: EV 充电也要买电, 是成本;
- $\epsilon_b(c + d)$: 很小的电池吞吐惩罚, 用来避免无意义的频繁充放电;
- $w_{dep}^{ev} g_i^2$: soft 模式下 EV 离家 SoC 不够时的惩罚。

这里的 g_i 是 EV 离家 SoC 缺口。它越大, 说明车离家时越没充够。

6.8 Local MPC 的优点和局限

Local MPC 的优点:

- 比 rule-based 更会利用未来电价、负荷和 PV 预测;
- 每个 agent 的优化问题比较小, 求解相对快;
- 行为容易解释, 因为它的目标函数和约束都很明确。

Local MPC 的局限:

- 每个 agent 只顾自己, 不知道其他 agent 同时在做什么;
- 多个用户可能同时在低价时充电, 造成变压器压力;
- 它不直接把完整电压、线路电流等 AC 电网约束写进优化问题。

7 ADMM MPC: 让多个用户边自私边协调

7.1 为什么需要 ADMM

假设一个小区里有 3 户家庭。每户都运行 Local MPC。某个时刻电价很低, 于是 3 户都决定给电池和 EV 充电。对每户来说, 这个决定都很合理; 但对整个小区变压器来说, 总功率可能太大。

这就是 Local MPC 的典型问题:

局部最优的动作，加在一起不一定是系统安全的动作。

ADMM MPC 的目的就是在不把所有用户完全合成一个巨大优化问题的情况下，让它们对系统级约束进行协调。本项目里主要协调的是变压器聚合功率。

7.2 先用故事理解 ADMM

ADMM 可以先不看公式，理解成三轮对话：

1. 每个家庭先各自做计划：我想在未来每一步用多少电；
2. 协调器检查总和：所有家庭加起来有没有超过变压器限制；
3. 如果超过了，就给每个家庭一个修正信号，让下一轮计划更接近安全总功率。

这个过程会重复多轮。每一轮家庭仍然关心自己的电费，但也会受到协调信号影响。最后得到的计划既考虑局部经济性，也尽量满足系统总功率约束。

7.3 ADMM 中三个新概念

ADMM MPC 会出现三个对初学者陌生的量：

- local variable: 每个 agent 自己算出来的净负荷计划；
- consensus variable: 协调后的公共副本，本文记为 $z_{i,k}$ ；
- dual variable: 历史修正信号，本文记为 $u_{i,k}^{admm}$ 。

其中 $z_{i,k}$ 不是 PV 弃光，PV 弃光仍然是 $u_{i,k}$ 。为了避免混淆，本文件把 ADMM 的对偶变量写成 $u_{i,k}^{admm}$ 。

可以把 u^{admm} 理解为“上几轮你偏离协调目标的累计提醒”。如果某个 agent 总是让总功率变得太高，这个提醒会在后续优化中推动它改变计划。

7.4 被协调的量：净负荷贡献

ADMM MPC 先定义每个 agent 的净负荷：

$$n_{i,k} = L_{i,t+k} - G_{i,t+k} + u_{i,k} + c_{i,k} - d_{i,k} + e_{i,k}. \quad (30)$$

当前实现中的贡献系数为：

$$\alpha_{i,k} = 1. \quad (31)$$

所以每个 agent 对变压器总功率的贡献就是：

$$q_{i,k} = \alpha_{i,k} n_{i,k} = n_{i,k}. \quad (32)$$

ADMM MPC 希望每个未来步都满足:

$$-\bar{P}^{trafo} \leq \sum_{i=1}^N q_{i,k} \leq \bar{P}^{trafo}. \quad (33)$$

其中:

$$\bar{P}^{trafo} = 0.97 P_{limit}^{trafo}. \quad (34)$$

这里的 0.97 是安全裕度。它表示不要把变压器刚好用到 100%，而是提前留一点余量。

7.5 局部子问题：每个 agent 怎样被协调

在 ADMM MPC 中，每个 agent 的优化目标从 Local MPC 的 J_i^{local} 变成:

$$\min J_i^{local} + \frac{\rho}{2} \|\alpha_i \odot \mathbf{n}_i - \mathbf{z}_i + \mathbf{u}_i^{admm}\|_2^2. \quad (35)$$

这个公式初看很吓人，但可以拆开:

- J_i^{local} : 自己原来的电费和 EV 惩罚;
- $\alpha_i \odot \mathbf{n}_i$: 自己计划对变压器的贡献;
- $\mathbf{z}_i$: 协调器希望它接近的贡献副本;
- $\mathbf{u}_i^{admm}$: 历史偏差带来的修正;
- ρ : 协调项有多强。

平方项的物理意义是：如果 agent 的计划和协调目标差得太远，它就会被惩罚。 ρ 越大，agent 越愿意为了系统协调牺牲自己的局部电费最优。

7.6 聚合投影：把总功率拉回安全范围

每一轮局部求解后，协调器拿到所有 agent 的贡献 $q_{i,k}$ 。它要保证:

$$-\bar{P}^{trafo} \leq \sum_i z_{i,k} \leq \bar{P}^{trafo}. \quad (36)$$

所有满足这个条件的 $\mathbf{z}$ 构成一个安全集合:

$$\mathcal{C} = \left\{ \mathbf{z} : -\bar{P}^{trafo} \leq \sum_i z_{i,k} \leq \bar{P}^{trafo}, \forall k \right\}. \quad (37)$$

投影可以写成:

$$\mathbf{z}^{new} = \Pi_{\mathcal{C}}(\mathbf{q} + \mathbf{u}^{admm}). \quad (38)$$

这里的“投影”可以理解成“找离原计划最近的安全计划”。如果总功率没有超过限制，投影前后基本不变。如果超过了限制，投影会把超出的部分分配回各个 agent 的副本，使总和落回安全区间。

例如某一步 3 个 agent 的贡献加起来是 110 kW，但安全上限是 100 kW，就超了 10 kW。项目实现中的思想是把这 10 kW 的超出量按简单方式分摊回各个 agent 的协调副本。这样下一轮每个 agent 再优化时，就会感受到“总功率太高，需要往下调”的信号。

7.7 对偶变量更新：记住偏差

投影之后，ADMM 更新对偶变量：

$$\mathbf{u}^{admm} \leftarrow \mathbf{u}^{admm} + \mathbf{q} - \mathbf{z}^{new}. \quad (39)$$

如果 $\mathbf{q}$ 和 $\mathbf{z}^{new}$ 相差很大，说明 agent 自己想要的计划和系统安全计划差距大。这个差距会被记进 $\mathbf{u}^{admm}$ ，影响下一轮局部优化。

所以 ADMM 不是一次命令所有人服从，而是通过多轮“计划、检查、修正、再计划”逐渐达成一致。

7.8 什么时候停止迭代

ADMM 用两个残差判断是否已经足够一致：

$$r^{pri} = \|\mathbf{q} - \mathbf{z}\|_2, \quad (40)$$

$$r^{dual} = \rho \|\mathbf{z}^{new} - \mathbf{z}^{old}\|_2. \quad (41)$$

原始残差 r^{pri} 衡量 agent 自己的计划和协调计划是否接近。对偶残差 r^{dual} 衡量协调变量本身是否还在大幅变化。

当二者都小到一定程度时：

$$r^{pri} \leq \epsilon^{pri}, \quad r^{dual} \leq \epsilon^{dual}, \quad (42)$$

算法就认为已经收敛。收敛不是说“宇宙真理已经找到”，而是说继续迭代带来的改变已经很小，可以停止了。

代码中还会做 residual balancing，也就是根据两个残差的相对大小调整 ρ 。直觉是：如果局部计划和协调计划差得太多，就加强协调；如果协调变量自己抖得太厉害，就减弱一点。

7.9 第一步动作投影：为什么还要再修一次

ADMM 算的是未来 H 步，但环境马上只执行第 1 步。因此项目代码在 ADMM 迭代后，还会对第一步动作做一个小的二次规划投影。

它的目标是：

$$\min \sum_i \left[(P_i^b - \hat{P}_i^b)^2 + (u_i - \hat{u}_i)^2 + (e_i - \hat{e}_i)^2 \right]. \quad (43)$$

其中带帽子的 $\hat{P}_i^b, \hat{u}_i, \hat{e}_i$ 是 ADMM 原本给出的动作。这个小问题的意思是：

尽量不要改太多，但必须把当前要执行的动作修到主要安全约束内。

特别是第一步变压器聚合功率要满足：

$$-\bar{P}^{trafo} \leq \sum_i n_{i,0} \leq \bar{P}^{trafo}. \quad (44)$$

这一步很重要，因为即使未来预测步还没有完全协调好，当前马上执行的动作也应该尽量满足主要聚约束。

7.10 ADMM MPC 的优点和局限

ADMM MPC 的优点：

- 比 Local MPC 多了变压器聚合功率协调；
- 不需要把所有 agent 完全合成一个巨大优化问题；
- 可以减少多个 agent 同时充电造成的系统峰值风险。

ADMM MPC 的局限：

- 当前主要协调变压器聚合功率，不等于完整 AC 潮流安全约束；
- 电压和线路约束没有像 Global MISOCP 那样完整进入优化；
- 有限迭代内可能还没完全收敛；
- 为了系统安全，可能牺牲一部分局部经济性。

8 soft 和 hard 模式：到底软在哪里，硬在哪里

8.1 先给核心直觉

soft 和 hard 主要讨论 EV 离家 SoC 要求。假设车早上要出门，希望离家时至少有 80% 电量。

soft 模式的态度是：

你最好达到 80%，如果没达到，就在目标函数里罚你。

hard 模式的态度是：

只要物理上还来得及，就必须保证离家目标；如果控制器给的充电太少，执行层会把它抬高。

因此：

- soft 是“允许违反，但违反要付代价”；
- hard 是“尽量不允许违反，必要时直接修正动作”。

8.2 soft 模式的数学表达

设离家步在当前预测窗口中的位置为 k_{dep} . EV 离家 SoC 缺口定义为：

$$g_i \geq SOC_{req}^{ev} - \frac{E_{i,k_{dep}}^{ev}}{C_i^{ev}}, \quad g_i \geq 0. \quad (45)$$

也可以写成更直观的形式：

$$g_i = \max\left(0, SOC_{req}^{ev} - SOC_{i,k_{dep}}^{ev}\right). \quad (46)$$

如果车离家时已经达到要求, $g_i = 0$ 。如果只充到 70%，而要求是 80%，那么 $g_i = 10\%$ 。soft 模式把缺口加入目标函数：

$$J \leftarrow J + w_{dep}^{ev} g_i^2. \quad (47)$$

平方 g_i^2 的意义是：小缺口会被罚，大缺口会被罚得更重。权重 w_{dep}^{ev} 越大，优化器越不愿意让 EV 离家电量不足。

soft 模式的好处是灵活，不容易因为要求太严格而导致优化问题不可行。坏处是它没有绝对保证：如果惩罚权重不够大，或者预测窗口没有看到离家时刻，EV 仍可能没充够。

8.3 hard 模式在本项目 MPC 中的实现

在标准优化教材里，hard 模式常常会直接写成硬约束：

$$E_{i,k_{dep}}^{ev} \geq SOC_{req}^{ev} C_i^{ev}. \quad (48)$$

这句话表示：离家时 EV 能量必须达到目标，不能通过“付罚金”逃避。

但本项目的 Local MPC 和 ADMM MPC 要特别区分：它们的局部优化问题本身仍然主要使用 soft 缺口结构。进入 hard 实验配置时，`scripts/mpc.py` 会启用环境执行层的 EV hard projection：

$$\text{ev_hard_projection_enabled}=\text{True}. \quad (49)$$

也就是说，对 Local MPC 和 ADMM MPC 来说，hard 模式的关键不是优化目标里多了一个神奇公式，而是环境在执行动作前会检查 EV 是否再不充就来不及。如果来不及，就把 EV 充电功率强制抬高到最低必要值。

8.4 hard projection 怎么算

设当前 EV SoC 为 $SOC_{i,t}^{ev}$ ，目标离家 SoC 为 SOC_{req}^{ev} 。环境会估计离家前还剩多少个可充电时间步，记为：

$$K_t. \quad (50)$$

如果未来每一步都用最大功率充电，未来最多还能增加：

$$\Delta SOC_{i,t}^{ev,max} = \frac{K_t P_i^{ev,max} \Delta t \eta_{ev}}{C_i^{ev}}. \quad (51)$$

如果现在 SoC 加上未来最多可补的 SoC 仍然不够，那么当前这一步就必须充一点。当前最低必要充电功率为：

$$P_{i,t}^{ev,min} = \max \left(0, \frac{(SOC_{req}^{ev} - SOC_{i,t}^{ev} - \Delta SOC_{i,t}^{ev,max}) C_i^{ev}}{\Delta t \eta_{ev}} \right). \quad (52)$$

控制器原本请求的 EV 充电功率是 $P_{i,t}^{ev,raw}$ 。hard projection 后实际执行：

$$P_{i,t}^{ev,exec} = \text{clip}(\max(P_{i,t}^{ev,raw}, P_{i,t}^{ev,min}), 0, P_i^{ev,max}). \quad (53)$$

这条公式可以翻译成三句话：

- 如果控制器请求已经足够大，就按控制器的来；
- 如果控制器请求太小，而再不充就来不及，环境会抬高到 $P_{i,t}^{ev,min}$ ；
- 最后还要限制在 0 到 $P_i^{ev,max}$ 之间，不能超过物理最大功率。

8.5 soft 和 hard 行为差异

soft 模式下，MPC 会在两件事之间权衡：

$$\text{现在少充电省钱} \quad \text{vs.} \quad \text{离家 SoC 不足被罚}. \quad (54)$$

hard 模式下，控制器可以等待便宜电价，但只能等到“不再等也来得及”的边界：

$$\text{可以等待低价} \quad \text{直到} \quad \text{再等就无法按时达标}. \quad (55)$$

所以 hard 模式通常会带来：

- EV 离家 SoC 满足率更稳定；
- 对惩罚权重不那么敏感；
- 接近离家时可能出现强制补电；
- 如果低价窗口已经错过，强制补电可能发生在高价时段；
- EV 强制充电可能和电池套利、变压器约束产生冲突。

9 Local MPC 和 ADMM MPC 在 soft/hard 下的对比

项目	Local MPC	ADMM MPC
基本思想	每个 agent 独立省钱	每个 agent 省钱, 同时通过 ADMM 协调总功率
未来信息	使用未来价格、负荷、PV 预测	使用未来价格、负荷、PV 预测
电池约束	有 SoC 和功率约束	有 SoC 和功率约束
PV 处理	PV 是预测输入, 可通过弃光变量削减	PV 是预测输入, 可通过弃光变量削减
EV soft	离家缺口进入目标函数惩罚	离家缺口进入局部目标函数惩罚
EV hard	主要依赖环境 hard projection 修正执行动作	主要依赖环境 hard projection 修正执行动作
变压器约束	不显式协调多 agent 总功率	显式协调聚合变压器功率
电压/线路约束	不完整显式建模	不完整显式建模, 主要处理聚合功率
主要优点	简单、快、容易解释	更能处理多用户同时动作导致的聚合风险
主要风险	各自最优可能叠加成系统过载	协调可能未完全收敛, 也可能牺牲部分经济性

10 Global MISOCP 和两种 MPC 的关系

仓库中的 MISOCP 是一个更强的全局优化基线。它不像 Local MPC 那样每个用户分开算, 也不像 ADMM MPC 那样只协调聚合变压器功率, 而是把更完整的电网约束放进同一个优化模型。

它包含类似 DistFlow 的网络约束, 例如支路 (u, v) 上:

$$P_{\ell,k} = \sum_{m \in \mathcal{C}(v)} P_{m,k} + r_{\ell} \ell_{\ell,k} + p_{v,k}^{inj}, \quad (56)$$

$$Q_{\ell,k} = \sum_{m \in \mathcal{C}(v)} Q_{m,k} + x_{\ell} \ell_{\ell,k}, \quad (57)$$

$$v_{v,k} = v_{u,k} - 2(r_{\ell} P_{\ell,k} + x_{\ell} Q_{\ell,k}) + (r_{\ell}^2 + x_{\ell}^2) \ell_{\ell,k}. \quad (58)$$

二阶锥松弛可以写成:

$$P_{\ell,k}^2 + Q_{\ell,k}^2 \leq v_{u,k} \ell_{\ell,k}. \quad (59)$$

对初学者来说, 不需要马上掌握 MISOCP 的全部数学。只要先记住:

- Local MPC: 每户自己优化;
- ADMM MPC: 每户自己优化, 但协调总变压器功率;
- Global MISOCP: 中央调度器直接考虑更完整的电网物理约束。

11 从零基础角度总结

理解 MPC, 可以抓住四个问题:

1. 控制器能决定什么? 答案是电池充放电、EV 充电、PV 弃光;
2. 控制器不能违反什么? 答案是电池容量、充放电功率、EV 连接状态、SoC 上下界等物理约束;
3. 控制器想优化什么? 答案是未来电费、EV 离家缺口惩罚、少量电池动作惩罚;
4. 控制器有没有考虑别人? Local MPC 没有, ADMM MPC 通过变压器聚合功率考虑了一部分。

因此, 本项目中的 MPC 可以理解为:

每一步, 控制器拿着未来预测数据, 先在纸上安排未来几步怎么充电、放电和弃光; 然后只执行当前一步; 下一步再根据新状态重新安排。

soft 和 hard 的区别可以理解为:

soft 是“没充够就罚分”; hard 是“快来不及时直接把充电动作抬高”。

带着这个直觉再回头看公式, 公式就不再是一堆符号, 而是在描述真实系统里的能量流、费用和安全边界。